Laporan Tugas Besar 1

IF3170 Inteligensi Artifisial

Optimasi Pengisian Paket pada Truk Peti Kemas dengan Algoritma Local Search

Tahun Akademik 2026–2027



Disusun oleh:

Kelompok 37

Samuelson Dharmawan Tanuraharja – 13524001
Nicholas Wise Saragih Sumbayak – 13524037
Neswa Eka Anggara – 13524136

Laboratorium Inteligensi Artifisial
Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Daftar Isi

1 Pendahuluan **3**
1.1 Latar Belakang . . . 3
1.2 Rumusan Persoalan dan Tujuan . . . 3
1.3 Ruang Lingkup Persoalan . . . 3

2 Dasar Teori **4**
2.1 Optimasi dan *Local Search* . . . 4
2.2 *State*, *Neighbor*, dan *Search Landscape* . . . 4
2.3 *Hill Climbing* . . . 5
2.4 *Simulated Annealing* . . . 5
2.5 *Genetic Algorithm* . . . 6

3 Pembahasan dan Implementasi **7**
3.1 Arsitektur Aplikasi . . . 7
3.1.1 Struktur Aplikasi . . . 7
3.1.2 Teknologi yang Digunakan . . . 7
3.2 Pemodelan Problem dan Constraint . . . 8
3.2.1 Data Problem . . . 8
3.2.2 Representasi State dan Orientasi . . . 9
3.2.3 Validasi Constraint . . . 9
3.2.4 Objective Function . . . 11
3.3 Implementasi Algoritma *Local Search* . . . 12
3.3.1 Inisialisasi State dan Pembangkitan Neighbor . . . 12
3.3.2 Hill Climbing . . . 12
3.3.3 Simulated Annealing . . . 12
3.3.4 Genetic Algorithm . . . 12
3.4 Rancangan Eksperimen . . . 13
3.4.1 Data Uji dan Lingkungan Eksperimen . . . 13
3.4.2 Konfigurasi Parameter dan Pengulangan Eksperimen . . . 13
3.5 Hasil Eksperimen dan Visualisasi Program . . . 14
3.5.1 Hill Climbing . . . 14
3.5.2 Simulated Annealing . . . 14
3.5.3 Genetic Algorithm . . . 14
3.5.4 Visualisasi State Awal dan State Akhir . . . 14
3.6 Analisis Eksperimen . . . 14
3.6.1 Perbandingan Kualitas Solusi . . . 14
3.6.2 Perbandingan Durasi Pencarian . . . 14
3.6.3 Konsistensi Hasil Eksperimen . . . 14
3.6.4 Pengaruh Parameter Genetic Algorithm . . . 14

4 Kesimpulan dan Saran **15**
4.1 Kesimpulan . . . 15
4.2 Saran . . . 15

5 Pembagian Tugas Tiap Anggota Kelompok **15**

Daftar Pustaka **15**

A Form Penggunaan Generative AI **15**

B Lampiran Lainnya (Bila Ada) **15**

1 Pendahuluan

1.1 Latar Belakang

Mejiro Express ingin mengatur pengiriman paket dengan truk peti kemas. Special Week, pengemudinya, perlu segera memutuskan paket mana yang akan dibawa dan bagaimana paket itu disusun. Setiap paket punya ukuran, nilai, berat, dan sifat kerapuhan yang berbeda. Susunannya harus muat di dalam peti kemas, stabil, dan tidak melebihi kapasitas angkut [1].

Pilihan posisi dan orientasi bertambah seiring banyaknya paket. Bahkan setelah paket yang akan dibawa dipilih, masih ada banyak cara untuk menatanya. Tugas ini memakai ***local search*** (pencarian lokal) untuk mencari susunan bernilai tinggi melalui perubahan kecil pada susunan yang sudah ada.

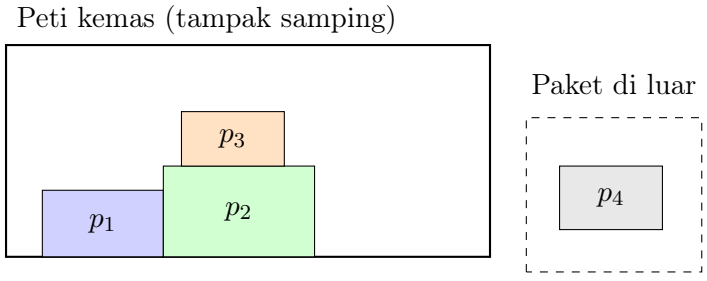
1.2 Rumusan Persoalan dan Tujuan

Masukannya berupa paket berbentuk balok atau kubus serta peti kemas dengan ukuran dan kapasitas angkut tertentu. Program harus memilih paket yang dimuat, lalu menentukan posisi dan orientasi tiap paket tersebut. Targetnya adalah memaksimalkan ***objective function*** (fungsi objektif), yaitu total nilai paket yang berhasil dimuat dalam susunan yang sah. Paket yang tidak dimuat tetap tercatat di luar peti kemas [1].

Pekerjaan ini menerapkan sedikitnya satu varian *hill climbing*, *simulated annealing*, dan *genetic algorithm*. Hasilnya dibandingkan dari nilai akhir, perubahan nilai selama pencarian, dan waktu yang dibutuhkan. Setiap percobaan juga diulang agar perbedaan hasil akibat pilihan awal acak dapat terlihat [1].

1.3 Ruang Lingkup Persoalan

Posisi paket memakai koordinat bilangan bulat pada ruang tiga dimensi. Paket dapat diputar sebesar 90° terhadap sumbu ruang. Setiap paket yang dimuat harus berada sepenuhnya di dalam peti kemas, tidak bertumpang tindih, dan memiliki penopang. Paket rapuh tidak boleh menopang paket lain. Total berat muatan juga tidak boleh melampaui kapasitas truk. Paket yang tidak dimuat berada sepenuhnya di luar peti kemas [1].



Gambar 1: Ilustrasi konseptual paket termuat dan paket di luar peti kemas. Susunan sebenarnya ditentukan dalam tiga dimensi.

2 Dasar Teori

2.1 Optimasi dan *Local Search*

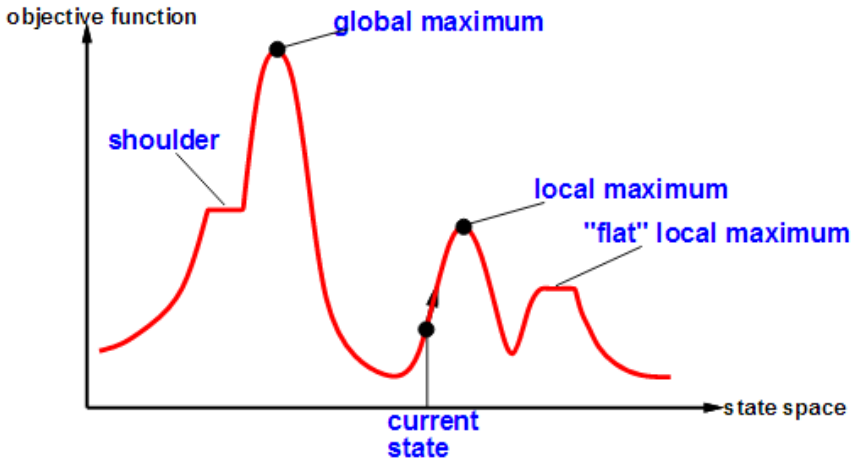
Dalam optimasi, *objective function* memberi nilai pada setiap kandidat solusi. Pencarian diarahkan ke kandidat dengan nilai setinggi mungkin. Jika ada ***constraint*** (batasan), kandidat yang dapat dipilih harus memenuhi constraint tersebut. Dengan $\mathcal{F}$ sebagai himpunan kandidat yang sah, tujuan umum masalah maksimisasi dapat ditulis $\max_{s \in \mathcal{F}} f(s)$, dengan $f(s)$ sebagai nilai kandidat s [2].

Local search bekerja pada satu kandidat lengkap dan memperbaikinya melalui perubahan bertahap. Yang menjadi hasil adalah konfigurasi akhirnya. Urutan perubahan selama pencarian tidak ikut menentukan nilai solusi. Pendekatan ini berguna ketika ruang kemungkinan terlalu besar untuk diperiksa satu per satu [2, 3].

2.2 *State*, *Neighbor*, dan *Search Landscape*

State (keadaan) adalah satu kandidat solusi yang lengkap. Pencarian dimulai dari *initial state* (state awal), yang bisa dibangkitkan secara acak. Sebuah *move* (langkah perubahan) menghasilkan ***neighbor*** (tetangga), yaitu state yang berbeda dalam sebagian kecil komponennya. Kumpulan neighbor yang dapat dicapai menentukan ke mana pencarian bisa bergerak dari state saat ini [3].

Nilai setiap state dapat dibayangkan sebagai ketinggian dalam *search landscape* (lanskap pencarian). ***Local maximum*** (puncak lokal) bernilai setidaknya sama tinggi dengan semua neighbor-nya, meskipun mungkin ada state lain yang nilainya lebih tinggi. Nilai tertinggi di seluruh ruang pencarian disebut *global maximum* (puncak global). Ada pula *plateau* (daerah datar), yaitu kumpulan state bertetangga dengan nilai yang sama. Local maximum dan plateau menjelaskan mengapa pencarian yang selalu mengejar kenaikan nilai bisa berhenti terlalu cepat [2, 3].



Gambar 2: *Search landscape* dengan global maximum, local maximum, dan plateau. Diambil dari salindia *Beyond Classical Search* [3].

2.3 *Hill Climbing*

Seperti terlihat pada Gambar 2, pencarian yang hanya memilih kenaikan nilai dapat berhenti di local maximum. ***Hill climbing*** memilih neighbor dengan nilai yang lebih baik daripada state saat ini. Dalam *steepest-ascent hill climbing*, semua neighbor yang tersedia dievaluasi, lalu algoritma mengambil yang terbaik. *Stochastic hill climbing* mengambil sampel neighbor secara acak dan berpindah jika nilainya naik. Kedua varian ini mudah terhenti di local maximum karena move yang menurunkan nilai ditolak [2, 3].

Hill climbing with sideways move juga menerima move ke neighbor dengan nilai sama. Jumlah move seperti ini biasanya dibatasi supaya pencarian tidak terus berputar di plateau. Cara lain adalah *random-restart hill climbing*, yang menjalankan pencarian beberapa kali dari initial state berbeda. Setiap restart memberi kesempatan untuk mencapai bagian search landscape yang belum terjelajahi [2, 3].

2.4 *Simulated Annealing*

Hill climbing tidak selalu bisa keluar dari local maximum dengan move yang hanya memperbaiki nilai. ***Simulated annealing*** sesekali menerima move yang menurunkan nilai agar pencarian dapat berpindah ke daerah lain. Peluang menerima penurunan ditentukan oleh *temperature* (temperatur). Untuk $\Delta f = f(s') - f(s) < 0$ dan $T > 0$, peluang menerima neighbor s' adalah

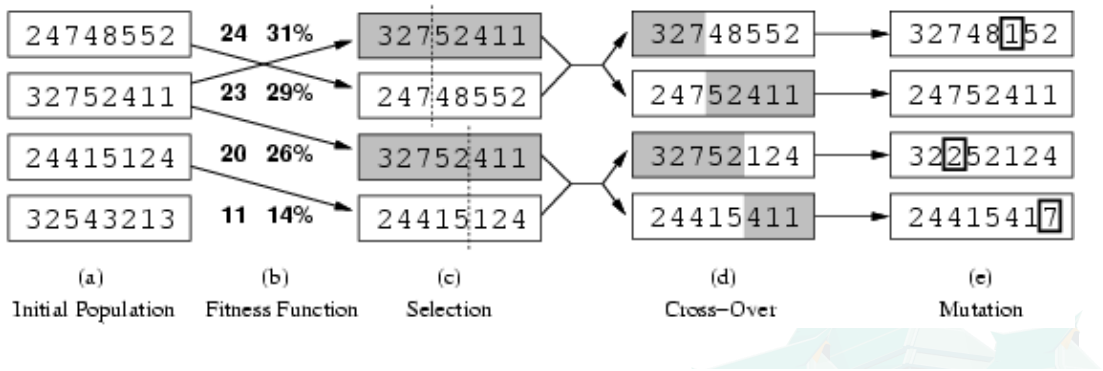
$$\Pr(\text{accept } s') = \exp\left(\frac{\Delta f}{T}\right). \tag{1}$$

Pada temperature tinggi, algoritma lebih sering menerima penurunan nilai sehingga dapat keluar dari local maximum. Seiring temperature turun, peluang itu mengecil. *Cooling schedule* (jadwal penurunan temperatur) menentukan seberapa cepat perilaku pencarian berubah. Jadwal yang terlalu cepat dapat membuat pencarian kembali terjebak sebelum sempat menjelajahi state lain [2, 3].

2.5 *Genetic Algorithm*

Jika hill climbing dan simulated annealing mengubah satu state dalam satu lintasan pencarian, ***genetic algorithm*** bekerja dengan *population* (populasi) yang berisi sejumlah *individual* (kandidat solusi). Setiap individual dinilai memakai *fitness function* (fungsi penilaian). Melalui *selection* (pemilihan induk), individual yang lebih baik mendapat peluang lebih besar untuk menghasilkan keturunan [2, 3].

Generasi baru dibentuk melalui *crossover* (penggabungan bagian dari dua induk) dan *mutation* (perubahan acak pada individual). Crossover mewariskan bagian dari solusi yang sudah ada, sementara mutation membantu menjaga keragaman population. Ukuran population dan banyaknya *generation* (generasi) memengaruhi jumlah kandidat yang diperiksa serta waktu pencarian. Bila suatu operator menghasilkan kandidat yang melanggar constraint, kandidat itu perlu ditangani sebelum dipakai sebagai solusi [2, 3].



Gambar 3: Alur selection, crossover, dan mutation pada genetic algorithm. Diambil dari salindia *Beyond Classical Search* [3].

3 Pembahasan dan Implementasi

3.1 Arsitektur Aplikasi

Aplikasi ini memiliki dua bagian yang berkomunikasi melalui HTTP. **Frontend** menerima konfigurasi problem dan menampilkan hasil pencarian, sedangkan **backend** memeriksa data, menjalankan algoritma, dan menghitung skor. Hasil pencarian berasal dari backend, sehingga keputusan apakah sebuah susunan sah tetap berada di satu tempat, yaitu validator backend.

3.1.1 Struktur Aplikasi

Kode frontend berada di `src/frontend/src`, sementara layanan dan algoritma berada di `src/backend/app`. Tabel 1 menunjukkan modul yang paling dekat dengan alur pengguna, mulai dari pengisian problem sampai hasil eksperimen tampil kembali.

Lokasi	Tanggung jawab
`frontend/src/components`	Form problem, kontrol algoritma, grafik, dan tampilan peti kemas tiga dimensi.
`frontend/src/api`	Menyusun request dan membaca respons dari backend.
`frontend/src/stores`	Menyimpan hasil eksperimen yang sedang dilihat dan posisi replay iterasi.
`backend/app/api`	Endpoint untuk validasi problem, menjalankan pencarian, batch, dan membaca hasil tersimpan.
`backend/app/domain`	Model paket, truk, problem, state, orientasi, dan move.
`backend/app/optimization`	Inisialisasi state, pembangkitan neighbor, validator, objective, dan algoritma pencarian.
`backend/app/experiments`	Runner, metrik, batch, dan penulisan hasil eksperimen.
`data` dan `output`	Berkas masukan serta hasil run yang disimpan jika opsi persistensi diaktifkan.

Tabel 1: Struktur utama aplikasi

Saat pengguna mengubah paket atau truk, frontend mengirim problem ke endpoint validasi. Untuk menjalankan pencarian, frontend mengirim problem, pilihan algoritma, parameter, dan *seed* ke endpoint eksperimen. `runner.py` membentuk generator acak dari seed tersebut, menjalankan algoritma, lalu memeriksa kembali state dan skor yang dikembalikan. Responsnya memuat state awal, state akhir, state terbaik, waktu proses, dan riwayat iterasi. Frontend menggunakan riwayat itu untuk replay dan grafik. Hasil juga dapat ditulis ke `output/experiments` bila persistensi diminta.

3.1.2 Teknologi yang Digunakan

Backend memakai **Python 3.12** dan **FastAPI** untuk API. **Pydantic** mendefinisikan model serta memeriksa input, sedangkan **NumPy** dipakai untuk occupancy grid tiga dimensi. Di sisi frontend, aplikasi dibuat dengan **React**, **TypeScript**, dan **Vite**. **React Three Fiber** bersama Three.js menampilkan paket dan peti kemas, **Recharts** menggambar perkembangan skor, dan **Zustand** menyimpan state replay.

3.2 Pemodelan Problem dan Constraint

Pemodelan dimulai dari data paket dan truk. Dari data itu, program membentuk state yang mencatat posisi setiap paket. State lalu melewati pemeriksaan constraint sebelum objective dihitung. Urutan ini dipakai bersama oleh seluruh algoritma, sehingga nilai yang dibandingkan selalu berasal dari **susunan yang sah**.

3.2.1 Data Problem

Kelas `Problem` menyimpan dua daftar: `PackageSpec` dan `TruckSpec`. Setiap paket memiliki ID, dimensi lebar, panjang, dan tinggi, kemudian `value`, `weight`, `is_fragile`, serta `eta`. Truk menyimpan ID, tiga dimensi ruang muat, dan kapasitas maksimum. Cuplikan 1 hingga 3 memperlihatkan hubungan ketiga kelas tersebut.

Cuplikan 1: Atribut paket pada `PackageSpec`

```
class PackageSpec(BaseModel):
    model_config = ConfigDict(frozen=True, allow_inf_nan=False)

    id: str
    width: int = Field(gt=0)
    length: int = Field(gt=0)
    height: int = Field(gt=0)
    value: float = Field(ge=0)
    weight: float = Field(ge=0)
    is_fragile: bool
    eta: int = Field(ge=0)
```

Cuplikan 2: Atribut truk pada `TruckSpec`

```
class TruckSpec(BaseModel):
    model_config = ConfigDict(frozen=True, allow_inf_nan=False)

    id: str
    width: int = Field(gt=0)
    length: int = Field(gt=0)
    height: int = Field(gt=0)
    max_capacity: float = Field(ge=0)
```

Cuplikan 3: Daftar paket dan truk dalam `Problem`

```
class Problem(BaseModel):
    packages: list[PackageSpec]
    trucks: list[TruckSpec]

    @model_validator(mode="after")
    def validate_ids_and_trucks(self) -> "Problem":
        if not self.trucks:
            raise ValueError("at least one truck is required")
        if len({package.id for package in self.packages}) != len(self.packages):
            raise ValueError("package IDs must be unique")
        if len({truck.id for truck in self.trucks}) != len(self.trucks):
            raise ValueError("truck IDs must be unique")
        return self
```

Validasi masukan dilakukan sejak model dibuat. Dimensi harus berupa bilangan bulat positif, sedangkan nilai, berat, ETA, dan kapasitas tidak boleh negatif maupun tak hingga.

`Problem` menolak ID paket atau truk yang berulang dan mensyaratkan setidaknya satu truk. Karena `trucks` berupa daftar, struktur yang sama dapat dipakai untuk satu truk maupun beberapa truk. Setelah data ini tersedia, penempatan setiap paket dicatat dalam state.

3.2.2 Representasi State dan Orientasi

Kelas `State` berisi daftar `PackagePlacement`. Pada state yang sah, **setiap paket memiliki tepat satu placement**. Di dalamnya tersimpan ID paket dan truk, `Position`, serta `Orientation`. Koordinat `Position` menunjuk sudut kiri-bawah-belakang paket. Sumbu x mengikuti lebar truk, y mengikuti panjang, dan z mengikuti tinggi. Paket di luar truk ditandai dengan `truck_id` dan `position` yang sama-sama bernilai `None`. Keduanya harus terisi bersama ketika paket dimuat, sebagaimana diperiksa oleh `validate_assignment` pada Cuplikan 4.

Cuplikan 4: Model penempatan paket

```
class PackagePlacement(BaseModel):
    package_id: str
    truck_id: str | None = None
    position: Position | None = None
    orientation: Orientation = Orientation.WLH

    @model_validator(mode="after")
    def validate_assignment(self) -> "PackagePlacement":
        if (self.truck_id is None) != (self.position is None):
            raise ValueError("truck_id and position must both be set or both be None")
        return self
```

Enam pilihan `Orientation` didefinisikan pada `app/domain/package.py`. Fungsi `get_oriented_dimensions` mengubah orientasi menjadi dimensi efektif (w_i, l_i, h_i) tanpa mengubah dimensi asli paket. Untuk kubus atau balok dengan sisi yang sama, beberapa orientasi menghasilkan dimensi efektif identik. Fungsi `get_unique_orientations` menghapus pilihan duplikat tersebut saat orientasi akan diacak. State yang terbentuk kemudian diperiksa sebagai satu kesatuan, bukan per paket saja.

3.2.3 Validasi Constraint

Fungsi `validate_state` pada `app/optimization/constraints.py` menjadi **pemeriksa akhir setiap kandidat**. Pemeriksaan dimulai dari kelengkapan assignment. Setiap paket dalam `Problem` harus muncul sekali dalam `State`, sedangkan ID paket dan truk yang tidak dikenal ditolak. Paket yang berada di luar tidak memakai ruang maupun kapasitas truk.

Validator membuat *occupancy grid* (grid keterisian) NumPy bagi setiap truk. Bentuk grid mengikuti (W_t, L_t, H_t), dengan sel kosong bernilai -1 dan sel terisi menyimpan indeks paket. Jika paket i ditempatkan di truk t pada (x_i, y_i, z_i) dengan dimensi efektif (w_i, l_i, h_i), seluruh bagian paket harus memenuhi

$$\begin{aligned} 0 \le x_i, \quad x_i + w_i \le W_t, \\ 0 \le y_i, \quad y_i + l_i \le L_t, \\ 0 \le z_i, \quad z_i + h_i \le H_t. \end{aligned} \tag{2}$$

Dalam kode, `_collect_occupancies` membandingkan posisi dan dimensi paket dengan ukuran grid. Cuplikan 5 menunjukkan pemeriksaan paket yang melewati dinding peti

kemas. Setelah lolos, irisan sel yang akan ditempati diperiksa lagi. Cuplikan 6 menolak paket ketika irisan tersebut sudah berisi indeks paket lain. Karena grid dibuat per truk, paket pada truk berbeda tidak saling bertabrakan.

Cuplikan 5: Pemeriksaan batas peti kemas

```
if any(
    start < 0 or start + size > limit
    for start, size, limit in zip(origin, dimensions, grid.shape, strict=True)
):
    errors.append(
        f"Package {placement.package_id} is out of bounds in truck {truck_id}"
    )
    unsafe_trucks.add(truck_id)
    continue
```

Cuplikan 6: Pemeriksaan overlap pada occupancy grid

```
for placement, package_index, region in regions:
    truck_id = placement.truck_id
    assert truck_id is not None
    cells = occupancies[truck_id][region]
    if np.any(cells != -1):
        errors.append(
            f"Package {placement.package_id} overlaps another in truck {truck_id}"
        )
        unsafe_trucks.add(truck_id)
        continue
    cells[...] = package_index
```

Paket pada $z_i = 0$ ditopang oleh lantai truk. Untuk paket di atas lantai, validator memeriksa sel tepat di bawah alasnya, yakni pada $z_i - 1$. Paket dinyatakan **melayang** jika tidak ada satu pun sel alas yang mendapat support dari paket lain. Jika ada support tetapi salah satu paket penopang bersifat rapuh, susunan juga ditolak. Pemeriksaan ini mengikuti definisi support pada tugas, yaitu cukup ada satu titik alas yang ditopang. Validator tidak mensyaratkan seluruh alas paket bersentuhan dengan penopang. Bagian inti pemeriksaan tersebut tampak pada Cuplikan 7.

Cuplikan 7: Pemeriksaan support dan paket rapuh

```
w, l, _ = get_oriented_dimensions(
    packages[placement.package_id], placement.orientation
)
x, y, z = position.x, position.y, position.z
below = occupancies[truck_id][x : x + w, y : y + l, z - 1]
supporters = np.unique(below[below >= 0])
if supporters.size == 0:
    support_errors.append(
        f"Package {placement.package_id} is floating in truck {truck_id}"
    )
elif any(problem.packages[int(index)].is_fragile for index in supporters):
    fragile_errors.append(
        f"Package {placement.package_id} rests on fragile support "
        f"in truck {truck_id}"
    )
```

Setelah posisi dan support diperiksa, validator menghitung beban setiap truk. Untuk

truk t, jumlah berat paket yang ditugaskan kepadanya harus memenuhi

$$\sum_{i \in P_t} \text{weight}_i \leq \text{max_capacity}_t. \tag{3}$$

Cuplikan 8 menunjukkan bahwa berat dijumlahkan dengan `Decimal`. Dengan cara ini, batas seperti $0.1 + 0.2 \leq 0.3$ tidak keliru akibat pembulatan bilangan *floating point*. Hasil `validate_state` berupa `ValidationResult` yang memuat status `valid`, daftar `errors`, dan total berat per truk. Jika assignment atau geometri sudah ambigu, pemeriksaan support dilewati karena grid tidak lagi dapat dipercaya, sementara kesalahan yang sudah ditemukan tetap dilaporkan.

Cuplikan 8: Penjumlahan berat dan pemeriksaan kapasitas

```
exact_weights = {truck.id: Decimal(0) for truck in problem.trucks}
for placement in loaded:
    assert placement.truck_id is not None
    exact_weights[placement.truck_id] += Decimal(
        str(packages[placement.package_id].weight)
    )
truck_weights = {
    truck_id: float(weight) for truck_id, weight in exact_weights.items()
}
for truck in problem.trucks:
    if exact_weights[truck.id] > Decimal(str(truck.max_capacity)):
        errors.append(
            f"Truck {truck.id} exceeds capacity: "
            f"{truck_weights[truck.id]} > {truck.max_capacity}"
        )
```

3.2.4 Objective Function

Setelah state dinyatakan sah, program menghitung nilainya melalui `ObjectiveMode` pada `app/optimization/objective.py`. **Mode utama**, `TOTAL_VALUE`, menjumlahkan `value` seluruh paket yang dimuat pada armada:

$$f_{\text{total}}(s) = \sum_{i:\, \text{truck}_i \neq \text{None}} \text{value}_i. \tag{4}$$

Mode tambahan, `ETA_WEIGHTED_VALUE`, memberi bobot lebih besar pada paket yang ETA-nya lebih dekat tanpa menambah atribut masukan:

$$f_{\text{ETA}}(s) = \sum_{i:\, \text{truck}_i \neq \text{None}} \frac{\text{value}_i}{\text{eta}_i + 1}. \tag{5}$$

Kontribusi setiap paket termuat dihitung oleh `objective_contributions`. Hasilnya dijumlahkan oleh `evaluate_objective` dengan `math.fsum`. Evaluator tidak memberikan penalti bagi pelanggaran constraint dan tidak memeriksa kelayakan state. Pemanggil harus memakai `validate_state` terlebih dahulu. Dengan begitu, **skor hanya dibandingkan di antara state yang sah**.

3.3 Implementasi Algoritma *Local Search*

3.3.1 Inisialisasi State dan Pembangkitan Neighbor

3.3.2 Hill Climbing

3.3.3 Simulated Annealing

3.3.4 Genetic Algorithm

3.4 Rancangan Eksperimen

3.4.1 Data Uji dan Lingkungan Eksperimen

3.4.2 Konfigurasi Parameter dan Pengulangan Eksperimen

3.5 Hasil Eksperimen dan Visualisasi Program

3.5.1 Hill Climbing

3.5.2 Simulated Annealing

3.5.3 Genetic Algorithm

3.5.4 Visualisasi State Awal dan State Akhir

3.6 Analisis Eksperimen

3.6.1 Perbandingan Kualitas Solusi

3.6.2 Perbandingan Durasi Pencarian

3.6.3 Konsistensi Hasil Eksperimen

3.6.4 Pengaruh Parameter Genetic Algorithm

4 Kesimpulan dan Saran

4.1 Kesimpulan

4.2 Saran

5 Pembagian Tugas Tiap Anggota Kelompok

Nama Lengkap	NIM	Tugas/Kontribusi
Samuelson Dharmawan Tanuraharja	13524001	
Nicholas Wise Saragih Sumbayak	13524037	
Neswa Eka Anggara	13524136	

Tabel 2: Pembagian tugas anggota kelompok

Daftar Pustaka

[1] Tim Asisten Laboratorium Inteligensi Artifisial, "Spesifikasi Tugas Besar 1 IF3170 Inteligensi Artifisial 2026/2027," ver. 1.4, Institut Teknologi Bandung, Bandung, Indonesia, Sep. 2026.

[2] S. J. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ, USA: Pearson, 2021.

[3] Tim Pengajar IF3170 Inteligensi Artifisial, "Beyond Classical Search," lecture slides, Institut Teknologi Bandung, Bandung, Indonesia, 2026.

A Form Penggunaan Generative AI

B Lampiran Lainnya (Bila Ada)